ARCHITECTURE

Full Runtime Architecture

CLIENTS

Order CLI
ecommerce/starter.py

Rewards CLI
rewards/starter.py

req →

TEMPORAL SERVER :7233

Frontend
gRPC based API

History Service
Event sourcing

Matching Service
Task routing

poll →

E-COMMERCE WORKER

process_payment

reserve_inventory

ship_package

poll →

REWARDS WORKER

send_welcome_email

send_tier_change

→

EXTERNAL SERVICES (mocked)

Payment Gateway
Stripe

Warehouse
Inventory API

Shipping Carrier
FedEx / UPS

Notifications
SendGrid / Twilio

Rewards Fulfillment

SAGA PATTERN

E-Commerce Order: Happy Path

Sequential activity chain with typed results flowing forward

1 process_payment

PaymentResult
{txn_id, amount}

→ PAYMENT_PROCESSED

→

2 reserve_inventory

InventoryResult
{reservation_id}

→ INVENTORY_RESERVED

→

3 ship_package

ShippingResult
{tracking_number}

→ SHIPPED

→

4 notify_customer

NotificationResult

→ COMPLETED

COMPENSATION

Saga Failure: Automatic Rollback

When shipping fails, all completed steps compensate in reverse order

FORWARD PATH

✓ `process_payment` → ✓ `reserve_inventory` → ✗ `ship_package`

After 3 retries
status = FAILED

⬇️ BEGIN COMPENSATION ⬇️

COMPENSATION PATH (reverse order)

↺ `release_inventory`

Releases res_abc123
Retry: up to 10×

← ↺ `refund_payment`

Refunds txn_xyz789
Retry: up to 10×

Final Status: COMPENSATED

Retry Policy Contrast

Forward steps: 3 retries

Fail fast, trigger compensation

Compensation: 10 retries

Try harder — must eventually succeed

ENTITY PATTERN

Rewards: Long-Lived Entity Workflow

Runs indefinitely as a stateful entity — reacting to events over time

1 Initialize

```
MembershipState
points=100, tier=BASIC
send_welcome_email()
```

→

2 Wait

```
wait_condition()
Parked — zero compute
Wakes when predicate true
```

→

3 Drain Loop

```
Process pending points
Process redemptions
Evaluate tier changes
```

→

4 Check

```
event_count ≥ 1000?
Continue-As-New
or back to Wait
```

↩ *Loop back to Wait (or Continue-As-New restarts at Init)*

SIGNAL HANDLERS

`add_points`

→ appends to _pending_points

`redeem_points`

→ appends to _pending_redemptions

`leave_program`

→ sets _should_leave = True

DEEP DIVE

The Drain Loop: Batch Signal Processing

Signals are received immediately but processed in batch — a critical design pattern

PARKED
wait_condition()

Workflow is idle.
Zero compute.
Waiting for predicate
to become true.

→

RECEIVE
Signal arrives

Handler fires.
Appends to queue.
Returns instantly.

→

RECEIVE
2nd signal

Also queued.
Both waiting
for drain.

→

DRAIN
Process batch

Pop signal #1 → update points,
evaluate tier → fire activity.
Pop signal #2 → update points,
evaluate tier.

→

RETURN
Back to wait

Queue empty.
Check threshold.
Re-park.